



UNIVERSITAS MULTI DATA PALEMBANG

Fakultas Ilmu Komputer dan Rekayasa

Program Studi Sistem Informasi

DOKUMENTASI BUKTI PEKERJAAN

Kuis Refactor: Arsitektur Backend Terstruktur

Route, Controller, Model, dan Middleware pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5C

Nama	Rizky Apriyandi Triadmojo
NPM	2428240107
Kelas	SI5C
Topik Tugas 1	Klinik: Dokter
Entitas utama	Dokter
Tautan repositori	https://github.com/rizkyapryndii/si5c_kuis1_rizkyapriyanditriadmojo
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	6 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah bukti bahwa Anda sendiri yang menyelesaikan *refactor* Tugas 1 ke arsitektur backend terstruktur. Dosen menilai dari dua hal: tangkapan layar (*screenshot*) yang membuktikan program berjalan, dan penjelasan Anda yang membuktikan bahwa Anda memahami kode tersebut.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya [doctors] dan [Entitas], dengan topik Tugas 1 Anda. Contoh: untuk topik Buku, [doctors] menjadi buku.
2. Setiap langkah memiliki tiga bagian: *screenshot*, keterangan gambar, dan kotak **Penjelasan**. Ketiganya wajib diisi.
3. Tulis penjelasan dengan kalimat Anda sendiri. Penjelasan yang hanya menyalin nama berkas atau tidak menyebut kode topik Anda dianggap belum mengerti.
4. Pada pengujian, gunakan data milik topik Anda dan masukkan identitas Anda (nama atau NIM) pada salah satu field teks pada permintaan POST. Ini menjadi tanda bahwa gambar berasal dari pekerjaan Anda.
5. Simpan hasil akhir sebagai PDF dengan nama PAW2_Kuis2_SI5C_2428240107_RizkyApyandiTriadmojo.pdf, lalu kumpulkan melalui LMS sesuai batas waktu yang ditentukan dosen.

B. Syarat tangkapan layar

Jenis	Yang harus terlihat
Terminal	Perintah yang diketik, seluruh keluarannya, dan nama folder project pada baris perintah.
Kode	Tidak perlu ditangkap layar. Kode dinilai dari tautan repositori GitHub.
Pengujian <i>endpoint</i>	Metode (GET, POST, PUT, DELETE), alamat lengkap, header yang dikirim, body (bila ada), kode status, dan isi respons. Boleh memakai Postman, Thunder Client, REST Client, atau curl di terminal.
Larangan	Gambar dipotong sehingga status atau isi respons tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Cara mengambil screenshot: Windows tekan Win + Shift + S; macOS tekan Cmd + Shift + 4; Linux gunakan PrtSc atau aplikasi tangkapan layar bawaan. Tempel gambar ke kotak pada dokumen ini, lalu atur lebarnya agar seluruh tulisan masih terbaca.

C. Susunan dokumen

Bab	Isi	Bukti utama
Bab 1	Instalasi	1 screenshot terminal
Bab 2	Struktur folder dan ringkasan kode	1 screenshot struktur folder, tabel penjelasan

Bab	Isi	Bukti utama
Bab 3	Menjalankan server	1 screenshot terminal
Bab 4	Pengujian endpoint (5 berhasil, 4 gagal)	9 screenshot
Bab 5	Rekapitulasi hasil uji	Tabel rekap
Bab 6	Repositori kode	Tautan GitHub
Bab 7	Refleksi	Dua jawaban tertulis

Seluruh dokumen hanya memerlukan **12 tangkapan layar**. Ambil tangkapan layar sambil mengerjakan, bukan di akhir, dan sisakan sekitar 25 menit dari waktu kuis untuk melengkapi penjelasan.

Identitas Topik dan Rancangan Endpoint

Bagian ini menjembatani topik Tugas 1 Anda dengan contoh yang dipakai pada soal. Isilah sebelum mengerjakan bab berikutnya. Seluruh tanda [...] pada bab selanjutnya mengacu ke tabel ini.

Data topik

Topik Tugas 1	Klinik: Dokter
Entitas utama (tunggal)	Dokter
Alamat dasar	/doctors
Field data	nama (string), spesialis (string), noStr (string), jadwalPraktik (string), biayaKonsultasi (number)
Field wajib saat POST	nama, spesialis, noStr
Query string untuk filter	GET /doctors?spesialis=Anak
Data awal	1: dr. Rina Kartika, Sp.A 2: dr. Budi Santoso, Sp.PD 3: dr. Siti Aulia, Sp.OG
Modul lain (jika ada)	Tidak ada

Daftar endpoint

Tuliskan seluruh *endpoint* pada API Anda. Kolom "API key" diisi Ya bila rute dilindungi cekApiKey.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/doctors	Ya	200	Ambil semua data (dengan filter spesialis bila ada)
2	GET	/doctors/:id	Ya	200	Ambil satu dokter berdasarkan ID
3	POST	/doctors	Ya	201	Tambah data dokter
4	PUT	/doctors/:id	Ya	200	Ubah data dokter
5	DELETE	/doctors/:id	Ya	204	Hapus data dokter

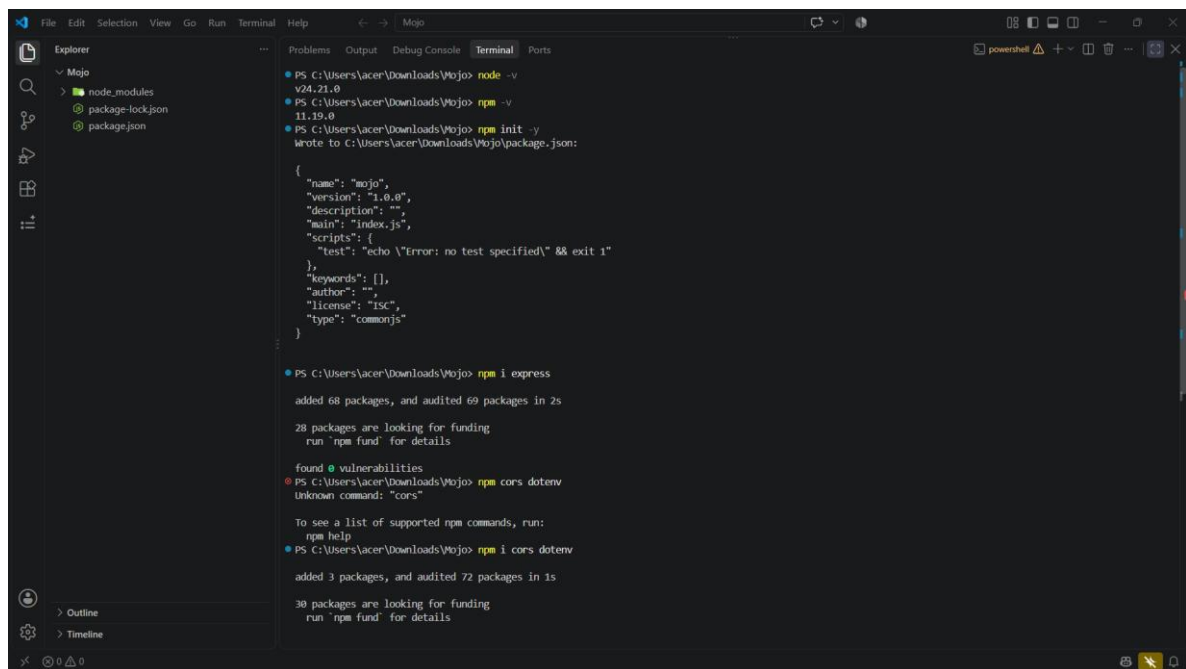
Jika topik Anda memiliki *endpoint* tambahan atau modul lain (misalnya relasi antartabel), tambahkan barisnya di sini dan uji pada bagian "Skenario Tambahan" di Bab 4.

Bab 1 Instalasi

Bab ini membuktikan bahwa lingkungan kerja siap dan *project* dibuat dari awal. Cukup **satu** tangkapan layar terminal yang memuat seluruh perintah instalasi.

1.1 Instalasi project

Harus terlihat pada screenshot: perintah `node -v`, `npm -v`, `npm init -y`, dan `npm install express cors dotenv` beserta keluarannya sampai selesai.



```
PS C:\Users\acer\Downloads\Mojo> node -v
v24.21.0
PS C:\Users\acer\Downloads\Mojo> npm -v
11.19.0
PS C:\Users\acer\Downloads\Mojo> npm init -y
wrote to C:\Users\acer\Downloads\Mojo\package.json:

{
  "name": "mojo",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

PS C:\Users\acer\Downloads\Mojo> npm i express

added 68 packages, and audited 69 packages in 2s

28 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\acer\Downloads\Mojo> npm cors dotenv
unknown command: "cors"

To see a list of supported npm commands, run:
  npm help
PS C:\Users\acer\Downloads\Mojo> npm i cors dotenv

added 3 packages, and audited 72 packages in 1s

30 packages are looking for funding
  run `npm fund` for details
```

Gambar 1. Versi Node.js dan instalasi paket

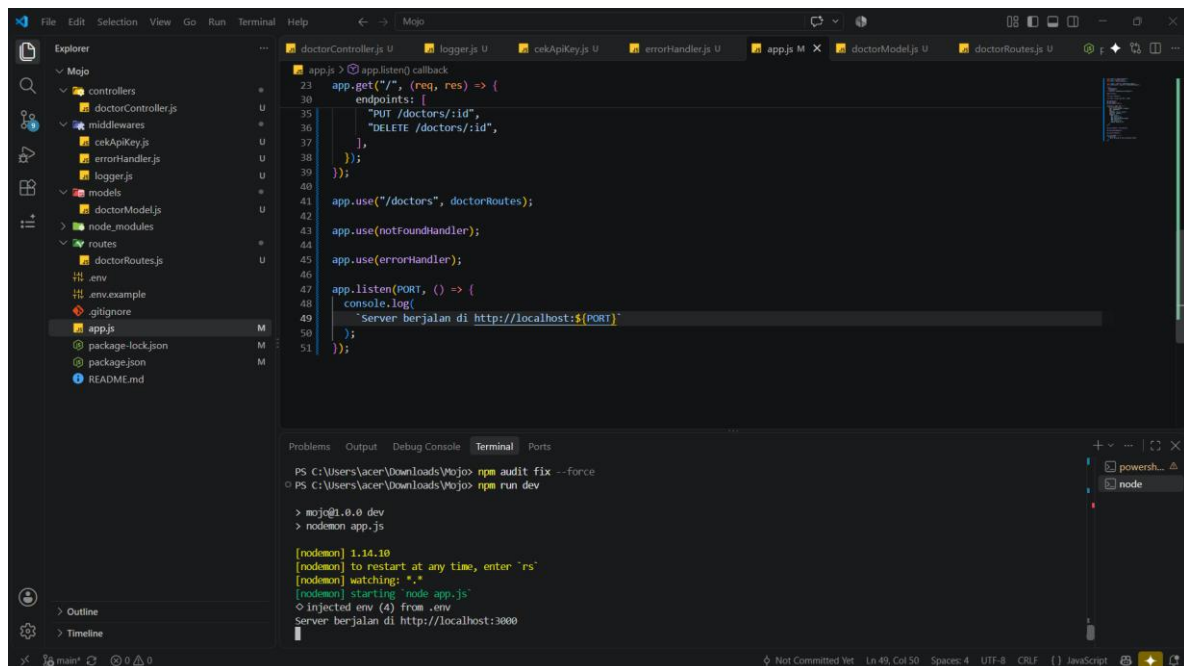
Penjelasan: Express digunakan untuk membangun server dan route RESTful API, cors mengizinkan request lintas origin, sedangkan dotenv membaca konfigurasi seperti PORT dan API_KEY dari berkas `.env`. Project menggunakan Node.js v24.21.0 dan Express 5 sehingga lingkungan runtime mendukung kebutuhan framework yang dipakai.

Bab 2 Struktur Folder dan Ringkasan Kode

Kode tidak perlu ditangkap layar. Dosen membaca kode langsung dari repositori (lihat Bab 6). Yang diminta di sini adalah **satu** tangkapan layar struktur folder dan penjelasan singkat tiap berkas dengan kalimat Anda sendiri.

2.1 Struktur folder

Harus terlihat pada screenshot: folder routes, controllers, models, dan middlewares beserta isinya. Gunakan panel Explorer editor atau perintah tree.



Gambar 2. Struktur folder project

Penjelasan: Folder models menyimpan data dan fungsi pengolahan dokter; controllers menangani request, validasi, dan response; routes memetakan endpoint /doctors ke Controller; middlewares berisi logger, cekApiKey, dan error handler untuk proses lintas request.

2.2 Ringkasan tugas setiap berkas

Tulis satu atau dua kalimat per berkas. Sebut nama fungsi atau variabel dari kode Anda.

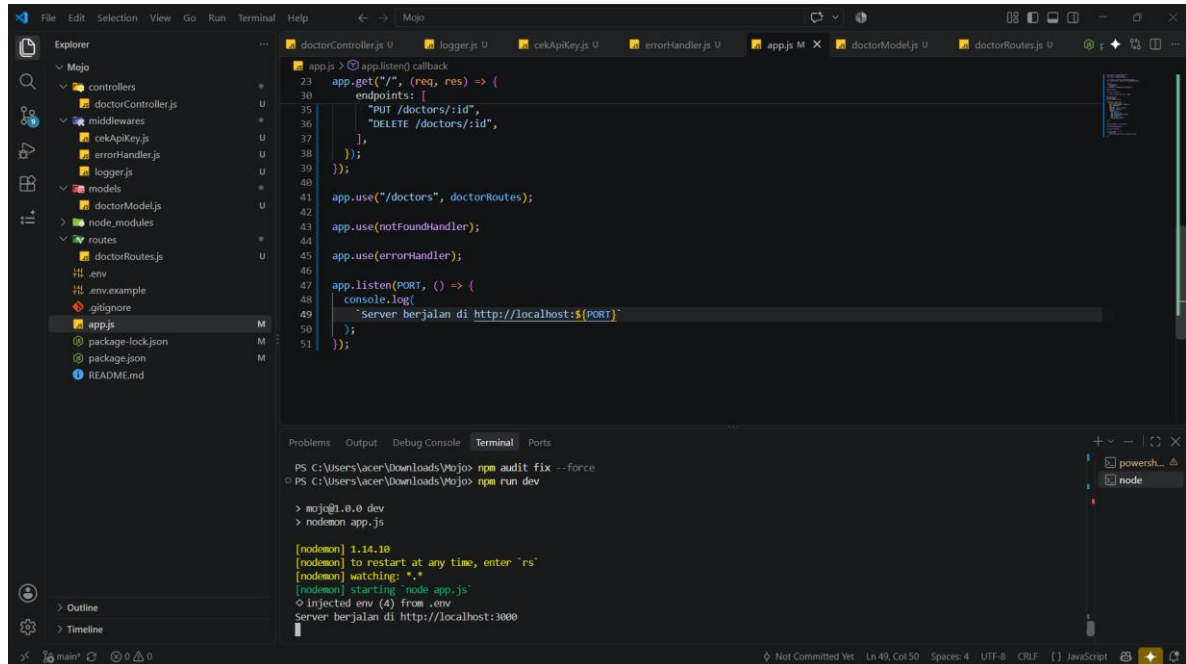
Berkas	Penjelasan Anda
models/doctorModel.js	Berisi array doctors dan fungsi getAllDoctors, getDoctorById, getDoctorsBySpesialis, createDoctor, updateDoctor, serta deleteDoctor. Model hanya mengelola data dan tidak menggunakan req/res.
controllers/doctorController.js	Berisi handler getAllDoctors, getDoctorById, createDoctor, updateDoctor, dan deleteDoctor serta validasi field wajib melalui validateDoctor. Controller membaca req dan mengirim response sesuai hasil proses.

Berkas	Penjelasan Anda
routes/doctorRoutes.js	Menggunakan <code>express.Router()</code> untuk memetakan GET /, GET /:id, POST /, PUT /:id, dan DELETE /:id ke Controller; POST, PUT, dan DELETE dilindungi <code>cekApiKey</code> .
middlewares/logger.js	Fungsi <code>logger</code> mencatat waktu, method, dan URL setiap request sebelum diteruskan dengan <code>next()</code> .
middlewares/cekApiKey.js	Fungsi <code>cekApiKey</code> mengambil header <code>x-api-key</code> lalu membandingkannya dengan <code>process.env.API_KEY</code> ; request tanpa key mendapat 401 dan key salah mendapat 403.
middlewares/errorHandler.js	<code>notFoundHandler</code> membuat error 404 untuk endpoint yang tidak ada, sedangkan <code>errorHandler</code> menangani error terpusat termasuk JSON rusak dan mengembalikan respons JSON yang konsisten.
app.js	Menginisialisasi Express, CORS, <code>dotenv</code> , <code>logger</code> , <code>express.json()</code> , route <code>/doctors</code> , <code>notFoundHandler</code> , dan <code>errorHandler</code> , lalu menjalankan server berdasarkan PORT dari <code>.env</code> .

Bab 3 Menjalankan Server

3.1 Menjalankan server

Harus terlihat pada screenshot: perintah `npm start` atau `npm run dev` dan pesan bahwa server berjalan beserta nomor port. Biarkan terminal ini tetap terbuka selama pengujian.



The screenshot shows the Visual Studio Code interface with a project named 'Mojo'. The Explorer sidebar on the left shows the file structure, including folders for 'controllers', 'middlewares', 'models', 'node_modules', and 'routes'. The main editor displays the 'app.js' file, which contains the following code:

```
23 app.listen() callback
24 app.get("/", (req, res) => {
25   endpoints: [
26     "PUT /doctors/:id",
27     "DELETE /doctors/:id",
28   ]
29 });
30
31 app.use("/doctors", doctorRoutes);
32
33 app.use(notFoundHandler);
34
35 app.use(errorHandler);
36
37 app.listen(PORT, () => {
38   console.log(
39     "Server berjalan di http://localhost:${PORT}"
40   );
41 });
```

The bottom panel shows the Terminal with the following output:

```
PS C:\Users\acer\Downloads\Vejoz> npm audit fix --force
PS C:\Users\acer\Downloads\Vejoz> npm run dev

> mojo@1.0.0 dev
> nodemon app.js

[nodemon] 1.14.10
[nodemon] to restart at any time, enter "rs"
[nodemon] watching: *.*
[nodemon] starting 'node app.js'
◇ injected env (4) from .env
Server berjalan di http://localhost:3000
```

Gambar 3. Server berhasil dijalankan

Penjelasan: Nomor port berasal dari `process.env.PORT` pada `.env` dan dipakai sebagai argumen `app.listen()`. Pada project ini server berjalan di port 3000. Jika port tersebut sudah dipakai aplikasi lain, proses Node.js tidak dapat melakukan bind ke port itu dan server perlu dijalankan pada port lain.

Bab 4 Pengujian Setiap Endpoint

Pengujian dibagi dua: lima skenario berhasil (T01 sampai T05) dan empat skenario gagal (N01 sampai N04). Skenario berhasil dilakukan **berurutan** karena T04 dan T05 memakai data yang dibuat pada T03. Setiap skenario cukup **satu** tangkapan layar yang memuat permintaan dan respons sekaligus; semua tulisan harus terbaca.

Catatan

Bunyi pesan boleh berbeda dengan contoh sesuai kode Anda, tetapi respons harus berupa JSON dan kode status harus tepat. Jika pesan Anda berbeda, tulis pesan yang sebenarnya pada kolom "Status hasil uji".

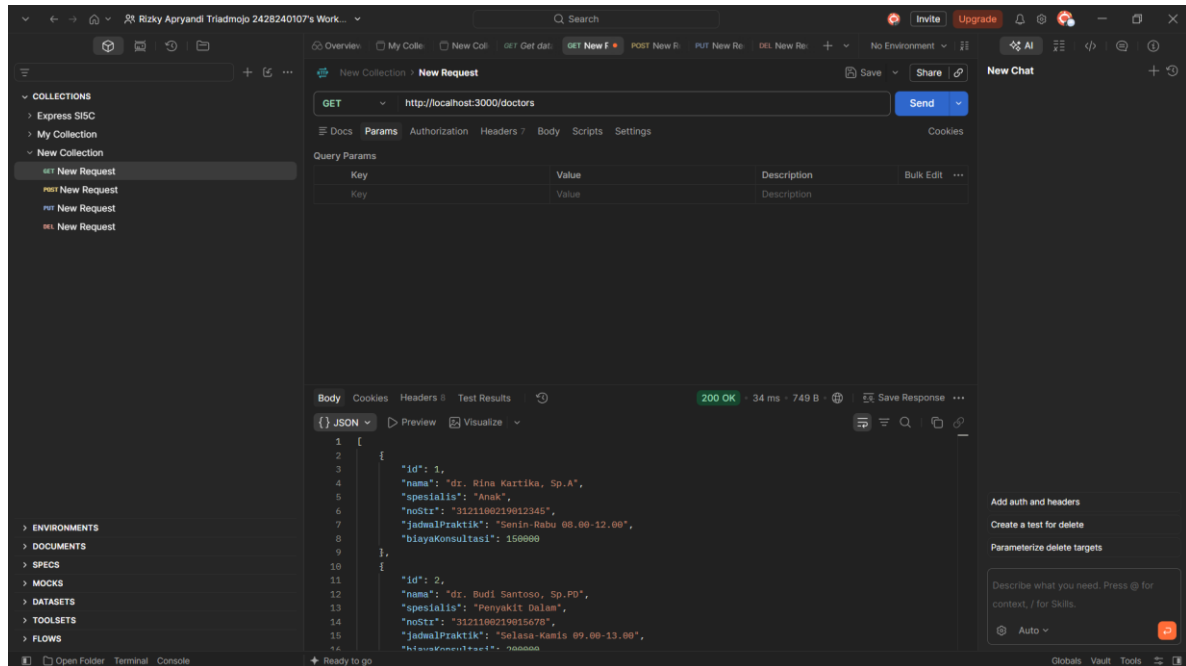
Gunakan ID 99 (atau ID lain yang pasti tidak ada) untuk skenario "tidak ditemukan".

Bagian A. Skenario berhasil

T01 Menampilkan semua data (GET /[doctors])

Harus terlihat pada screenshot: metode GET, alamat lengkap, kode status, dan isi respons berupa larik JSON.

Permintaan (<i>request</i>)	GET /doctors
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; isi berupa larik JSON seluruh data dokter.
Status hasil uji	200 OK
Lulus?	☒ Ya ☐ Tidak



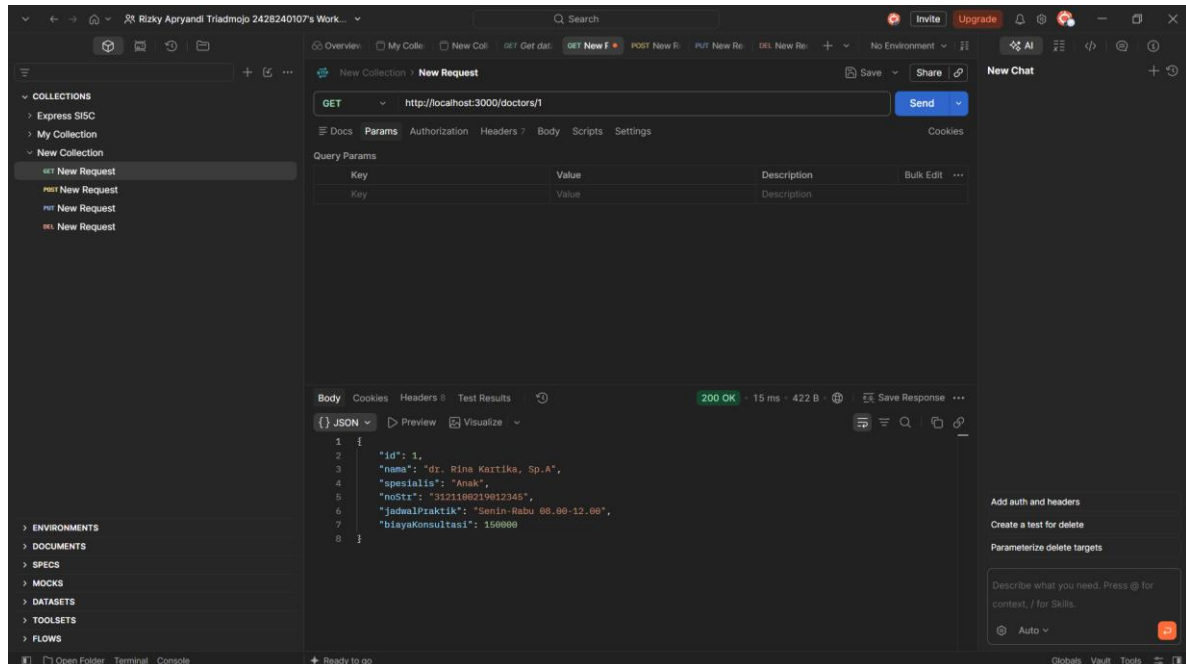
Gambar 4. Hasil GET semua data

Penjelasan: Status 200 menunjukkan request berhasil. Route /doctors meneruskan GET ke doctorController.getDoctors, lalu Controller memanggil doctorModel.getAllDoctors dan mengembalikan seluruh data dokter dalam bentuk larik JSON.

T02 Menampilkan satu data berdasarkan ID

Harus terlihat pada screenshot: ID pada alamat dan objek tunggal pada respons.

Pemintaan (request)	GET /doctors/1
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; satu objek JSON dengan id 1.
Status hasil uji	200 OK
Lulus?	[X] Ya [] Tidak



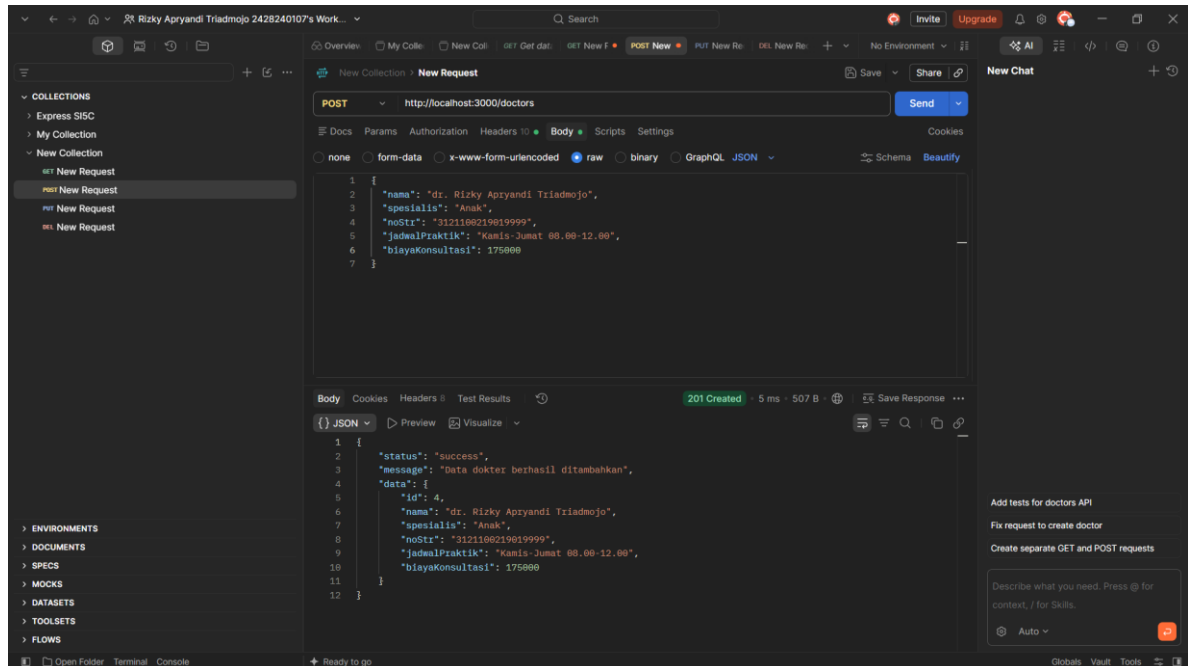
Gambar 5. Hasil GET berdasarkan ID

Penjelasan: Nilai ID diambil dari `req.params.id` pada route `/doctors/:id`. Karena `req.params` selalu berupa string, nilainya diubah menjadi number dengan `parseInt()` agar dapat dibandingkan dengan id numerik pada model.

T03 Menambah data baru (POST)

Harus terlihat pada screenshot: header `x-api-key` dan `Content-Type`, body JSON, status 201, serta objek baru lengkap dengan id. Gunakan data identitas Anda sendiri pada salah satu field teks (misalnya nama atau NIM).

Permintaan (request)	POST /doctors
Header	x-api-key: SI5C-KUIS1-DOCTOR-2428240107 Content-Type: application/json
Body	{ "nama": "dr. Rizky Apriyandi Triadmojo", "spesialis": "Anak", "noStr": "3121100219019999", "jadwalPraktik": "Kamis-Jumat 08.00-12.00", "biayaKonsultasi": 175000 }
Hasil yang diharapkan	201 Created; objek baru beserta id yang dibuat server.
Status hasil uji	201 Created — Data dokter berhasil ditambahkan
Lulus?	[X] Ya [] Tidak



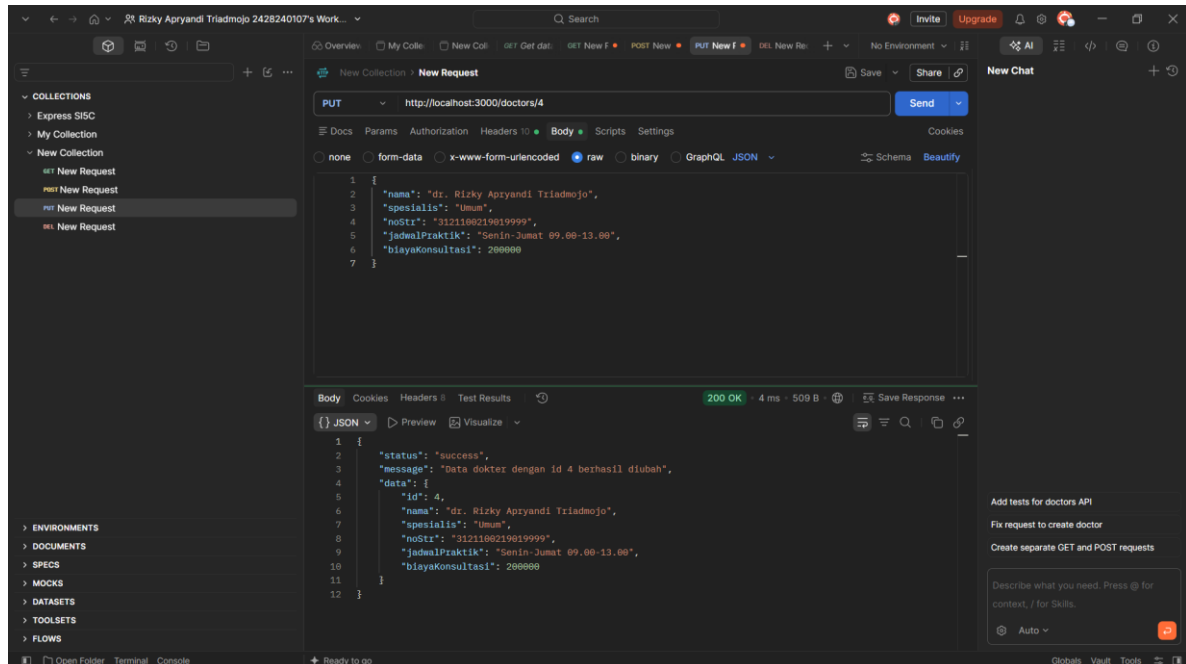
Gambar 6. Hasil POST data baru

Penjelasan: Header x-api-key diperiksa oleh middleware cekApiKey sebelum Controller dijalankan. `express.json()` mengubah body JSON menjadi object yang dapat dibaca melalui `req.body`. Status 201 dipakai karena server berhasil membuat resource dokter baru, bukan hanya mengambil atau mengubah data.

T04 Mengubah data (PUT)

Harus terlihat pada screenshot: ID data baru pada alamat, header lengkap, body berisi field yang diubah, dan objek hasil perubahan.

Permintaan (request)	PUT /doctors/4
Header	x-api-key: SI5C-KUIS1-DOCTOR-2428240107 Content-Type: application/json
Body	{ "nama": "dr. Rizky Apriyandi Triadmojo", "spesialis": "Umum", "noStr": "3121100219019999", "jadwalPraktik": "Senin-Jumat 09.00-13.00", "biayaKonsultasi": 200000 }
Hasil yang diharapkan	200 OK; objek dengan field yang sudah berubah.
Status hasil uji	200 OK — Data dokter dengan id 4 berhasil diubah
Lulus?	[X] Ya [] Tidak



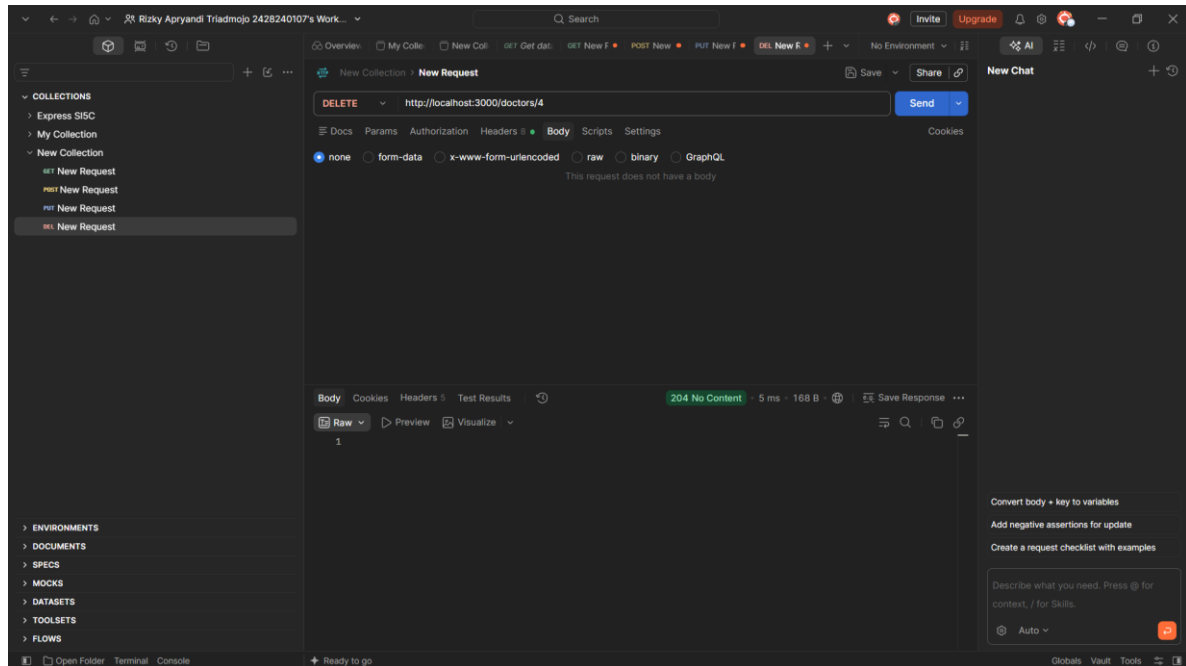
Gambar 7. Hasil PUT mengubah data

Penjelasan: Pada request PUT, data dokter ID 4 berubah pada field spesialis, jadwalPraktik, dan biayaKonsultasi. Nama dan noStr tetap sama. Perubahan diteruskan dari route ke `doctorController.updateDoctor` lalu dilakukan oleh `doctorModel.updateDoctor`.

T05 Menghapus data (DELETE)

Harus terlihat pada screenshot: metode DELETE, header x-api-key, status 204, dan respons tanpa isi.

Permintaan (request)	DELETE /doctors/4
Header	x-api-key: SI5C-KUIS1-DOCTOR-2428240107
Body	(tidak ada)
Hasil yang diharapkan	204 No Content; tanpa isi respons.
Status hasil uji	204 No Content
Lulus?	[X] Ya [] Tidak



Gambar 8. Hasil DELETE data

Penjelasan: Status 204 berarti penghapusan berhasil dan server tidak perlu mengirim representasi data baru. Karena resource dokter ID 4 sudah dihapus, respons sengaja tidak memiliki body.

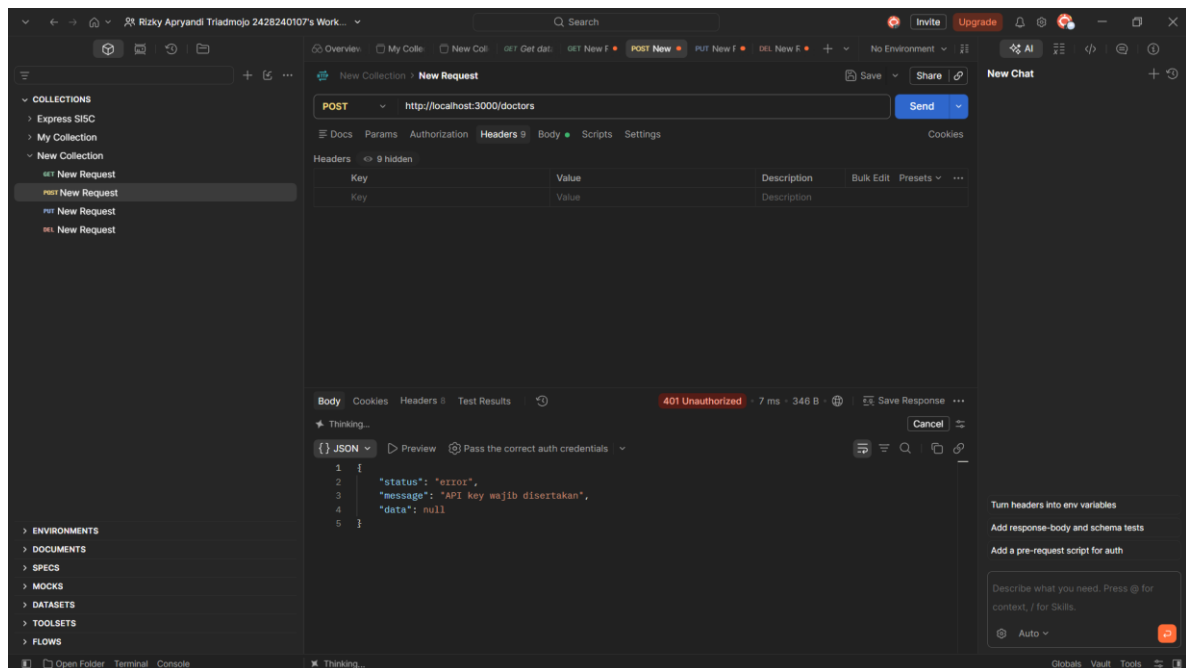
Bagian B. Skenario gagal

Skenario gagal membuktikan bahwa API menolak permintaan yang salah dengan kode status dan pesan yang tepat, bukan runtuh atau membalas halaman HTML berisi jejak error.

N01 Menambah data tanpa API key

Harus terlihat pada screenshot: permintaan POST tanpa header x-api-key dan respons 401.

Permintaan (<i>request</i>)	POST /doctors (tanpa x-api-key)
Header	Content-Type: application/json
Body	{ "nama": "Dokter Tanpa API Key", "spesialis": "Anak", "noStr": "3121100219018888", "jadwalPraktik": "Senin 08.00-12.00", "biayaKonsultasi": 150000 }
Hasil yang diharapkan	401 Unauthorized; pesan JSON; data tidak bertambah.
Status hasil uji	401 Unauthorized — API key wajib disertakan
Lulus?	[X] Ya [] Tidak



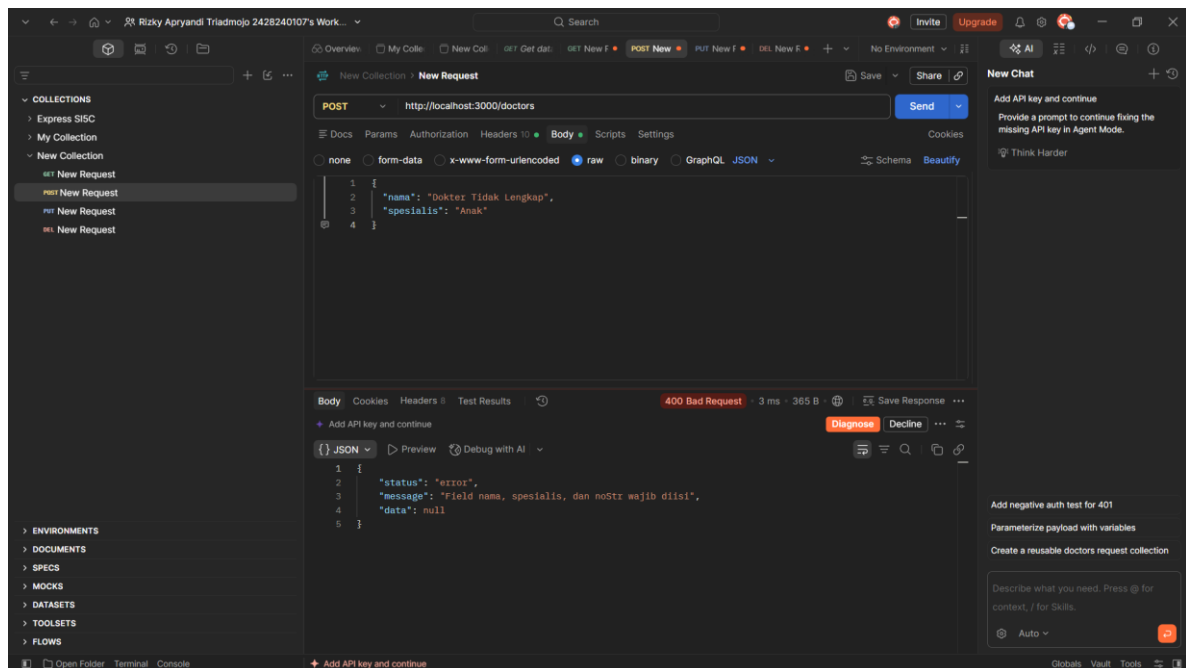
Gambar 9. POST tanpa API key ditolak

Penjelasan: Middleware cekApiKey langsung memeriksa header x-api-key. Karena header tidak ada, middleware mengembalikan 401 dan tidak memanggil next(), sehingga Controller createDoctor tidak pernah dijalankan dan data tidak bertambah.

N02 Menambah data dengan body tidak lengkap

Harus terlihat pada screenshot: body yang sengaja kekurangan field wajib dan respons 400.

Permintaan (<i>request</i>)	POST /doctors
Header	x-api-key: SI5C-KUIS1-DOCTOR-2428240107 Content-Type: application/json
Body	{ "nama": "Dokter Tidak Lengkap", "spesialis": "Anak" }
Hasil yang diharapkan	400 Bad Request; pesan menyebut field yang wajib.
Status hasil uji	400 Bad Request — Field nama, spesialis, dan noStr wajib diisi
Lulus?	[X] Ya [] Tidak



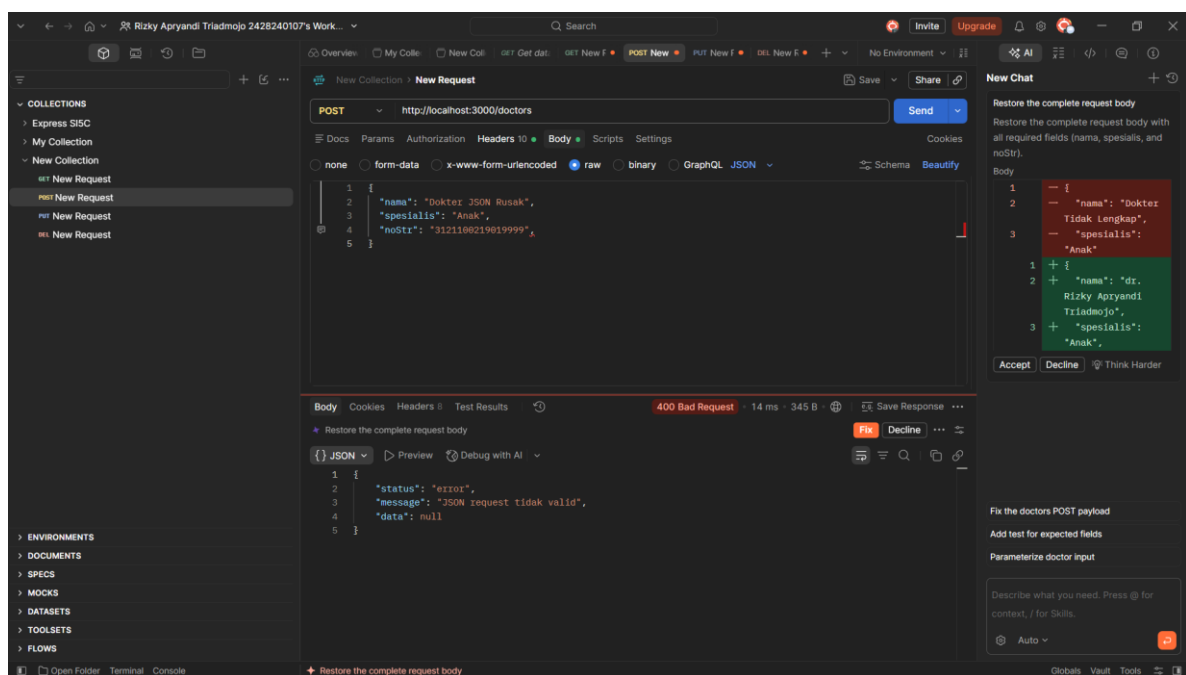
Gambar 10. POST dengan body tidak lengkap

Penjelasan: Validasi field wajib ditulis pada Controller melalui fungsi `validateDoctor`. Karena `noStr` tidak dikirim, Controller menghentikan proses dan mengembalikan `400 Bad Request` yang menunjukkan request tidak memenuhi data yang diwajibkan.

N03 Mengirim JSON yang rusak

Harus terlihat pada screenshot: body dengan tanda baca salah (misalnya koma berlebih) dan respons 400 berformat JSON.

Permintaan (<i>request</i>)	POST /doctors
Header	x-api-key: SI5C-KUIS1-DOCTOR-2428240107 Content-Type: application/json
Body	{ "nama": "Dokter JSON Rusak", "spesialis": "Anak", "noStr": "3121100219019999", }
Hasil yang diharapkan	400 Bad Request; pesan "Format JSON tidak valid" (bukan halaman HTML).
Status hasil uji	400 Bad Request — JSON request tidak valid
Lulus?	[X] Ya [] Tidak



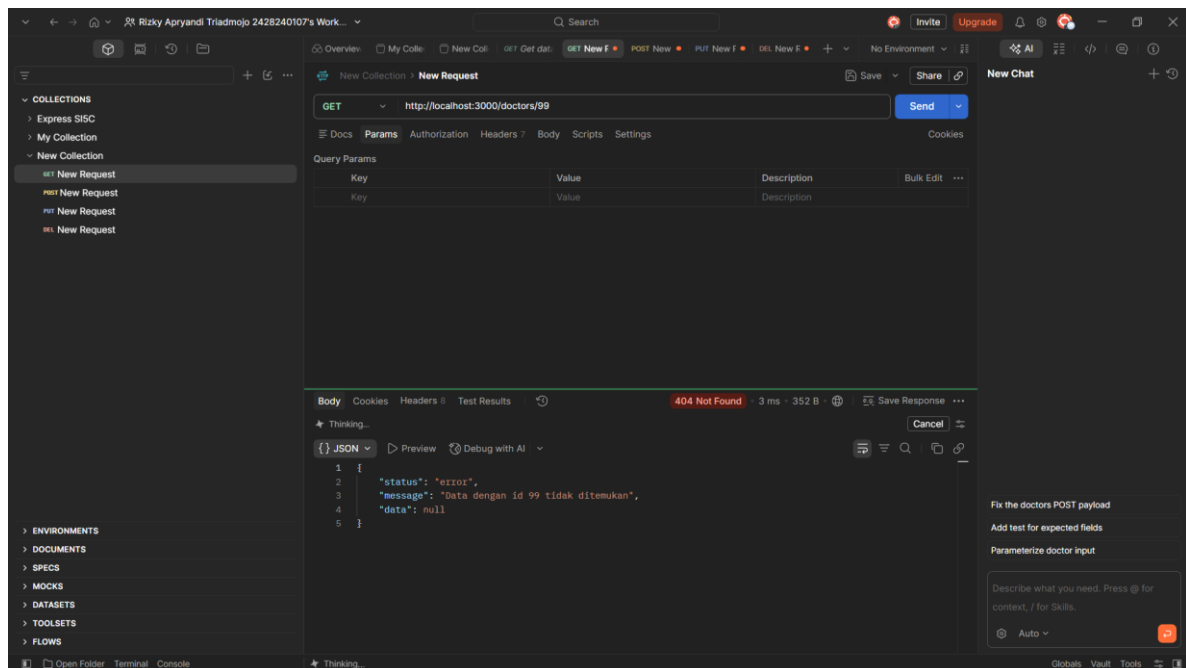
Gambar 11. POST dengan JSON rusak

Penjelasan: JSON parser pada `express.json()` mendeteksi sintaks body yang rusak sebelum request mencapai Controller. Error tersebut diteruskan ke `errorHandler`, lalu middleware mengubahnya menjadi respons JSON 400 yang rapi sehingga client tidak menerima halaman HTML.

N04 Mengambil ID yang tidak ada

Harus terlihat pada screenshot: ID yang jelas tidak ada (misalnya 99) dan respons 404.

Permintaan (request)	GET /doctors/99
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	404 Not Found; pesan JSON.
Status hasil uji	404 Not Found — Data dengan id 99 tidak ditemukan
Lulus?	[X] Ya [] Tidak



Gambar 12. GET ID tidak ada

Penjelasan: Route `GET /doctors/:id` memanggil `doctorController.getDoctorById`. Controller meminta `doctorModel.getDoctorById(99)`; karena hasilnya `null/undefined`, Controller membuat respons 404 dengan pesan bahwa ID 99 tidak ditemukan. Ini adalah error aplikasi untuk resource yang tidak tersedia.

Bab 5 Rekapitulasi Hasil Uji

Salin hasil setiap skenario ke tabel berikut. Tabel ini dibandingkan dosen dengan tangkapan layar pada Bab 4.

ID	Skenario	Status harapan	Status hasil	Lulus
T01	Menampilkan semua data	200	200 OK	Ya
T02	Menampilkan satu data berdasarkan ID	200	200 OK	Ya
T03	Menambah data baru	201	201 Created	Ya
T04	Mengubah data	200	200 OK	Ya
T05	Menghapus data	204	200 OK	Ya
N01	Menambah data tanpa API key	401	401 Unauthorized	Ya
N02	Menambah data dengan body tidak lengkap	400	400 Bad Request	Ya
N03	Mengirim JSON yang rusak	400	400 Bad Request	Ya
N04	Mengambil ID yang tidak ada	404	404 Not Found	Ya

Bab 6 Repositori Kode

Kode dinilai dari repositori GitHub, bukan dari tangkapan layar. Pastikan repositori dapat dibuka dosen (publik atau dosen diundang).

Tautan repositori	https://github.com/rizkyapryndii/si5c_kuis1_rizkyapriyanditria_dmojo
Pesan commit terakhir	Refactor API dokter ke arsitektur backend terstruktur
Folder routes, controllers, models, middlewares ada	☒ Ya ☐ Tidak
.env tidak terunggah (ada di .gitignore)	☒ Ya ☐ Tidak
.env.example ada	☒ Ya ☐ Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan kode topik Anda.

Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

Penjelasan: Route hanya memetakan alamat endpoint ke fungsi Controller, Controller menangani request, validasi, dan response, sedangkan Model mengelola data dokter. Contohnya `doctorRoutes.js` memanggil `doctorController.getDoctors`, lalu Controller mengambil data melalui `doctorModel.getAllDoctors`.

Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

Penjelasan: Error handler terpusat membuat format error lebih konsisten dan mengurangi pengulangan kode. Buktinya terlihat pada N03, ketika JSON rusak dari `express.json()` tetap diubah menjadi respons JSON 400 oleh errorHandler, sedangkan endpoint lain juga dapat menggunakan penanganan 404 yang seragam.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda [. . .] sudah diganti dengan topik Tugas 1 saya	Ya
2	Semua tangkapan layar berasal dari komputer saya dan terbaca jelas	Ya
3	Setiap gambar memiliki nomor, keterangan, dan penjelasan	Ya
4	Identitas saya (nama atau NIM) muncul pada data uji POST	Ya
5	Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji	Ya
6	Tabel rekapitulasi sesuai dengan tangkapan layar	Ya
7	Tautan repositori pada sampul dapat dibuka dan .env tidak terunggah	Ya
8	Berkas disimpan sebagai PDF dengan nama sesuai ketentuan	Ya

Lampiran B Rubrik Penilaian (Usulan)

Bobot di bawah ini adalah usulan awal dan dapat diubah dosen sebelum dibagikan ke mahasiswa.

Komponen	Kriteria penilaian penuh	Bobot
Instalasi dan struktur folder (Bab 1 sampai 3)	Instalasi, struktur folder, dan server berjalan benar	15
Ringkasan kode dan kode di repositori (Bab 2 dan 6)	Tanggung jawab tiap layer tidak tercampur; penjelasan menyebut kode sendiri; .env tidak terunggal	30
Pengujian skenario berhasil (T01 sampai T05)	Status dan respons sesuai; penjelasan benar	25
Pengujian skenario gagal (N01 sampai N04)	Kode status dan pesan JSON sesuai; penjelasan alur error benar	20
Rekapitulasi dan refleksi (Bab 5 dan 7)	Rekap konsisten dengan bukti; jawaban pribadi	10
Total		100